

Dual-Engine Local Reasoning

Hailo-10H planner in parallel with llama.cpp Qwen3.8-2B-Distill on the Raspberry Pi 5 host. Operator types one CLI sentence. The engine splits, overlaps, and merges without a second prompt.

Field	Value
Document	MN-ARCH-REASON-001
Title	Mercury CLI Hailo + CPU Split Reasoning Architecture
Audience	Luna (planning and implementation)
Status	Draft for implementation
Date	2026-09-02
Baseline repo	github.com/humanvalueexchange/mercury-node @ main
Live host snapshot	mercury2 reconciliation 2026-09-02
Classification	Internal engineering spec. No secrets, no keys.

This specification is written so an implementer can change the CLI, systemd units, and prompt contracts without rediscovering hardware limits. Every “do not” is load-bearing. The money layer (Bitcoin Core, LND) must keep working if either model process dies.

Contents

1. Purpose and operator contract
2. Non-goals and hard constraints
3. Current reference deployment
4. Why a split is required
5. Target architecture
6. Runners, ports, and models
7. Prompt contracts
8. Parallel control flow and UX
9. Snapshot builder (context the models see)
10. Sample libraries and CLI integration
11. systemd, affinity, and thermal budget
12. Failure modes and fallbacks
13. Security and money-layer isolation
14. Files to touch
15. Implementation plan for Luna
16. Acceptance tests
17. Worked example: channel rebalance recommendation
18. Open questions left to Luna

1. Purpose and operator contract

The operator continues to type a single Mercury CLI sentence. They do not name models, ports, Hailo, llama.cpp, planners, or merges. Example:

```
mercury ask "Recommend the best lightning channel for rebalancing inbound and outbound liquidity"
```

The engine must: collect live node snapshot; fire Hailo and CPU together; stream something useful within Hailo TTFT (~0.3–1.0 s if the HEF is warm); produce one final operator-facing answer that prefers on-node numbers over either model’s guesses; never require a second typed prompt; never move funds.

Invisible UX means: no “calling Hailo...”, no “draft vs plan” panels in v1, no spinner that admits two backends, no extra flags on the happy path. Debugging verbosity is an env flag, not a default.

1.1 What “simultaneous split” means

It does **not** mean one model sharded across the Hailo-10H and the four Cortex-A76 cores. Hailo-ollama executes an entire HEF graph on the NPU. llama-server executes an entire GGUF on the CPU. There is no official layer-split, tensor-parallel, or speculative-decode bridge between those two runtimes.

It **does** mean two completions start at the same wall-clock time on the same operator question plus the same snapshot. Hailo returns a short JSON plan. CPU returns a short prose draft. A second, cheap CPU call merges them. Wall time ≈ max(plan, draft) + merge, not the sum of three serial model calls.

2. Non-goals and hard constraints

- Do not introduce vLLM. There is no Hailo backend and no realistic Pi 5 deployment path.
- Do not run stock Ollama on the Pi as a third llama.cpp. mercury-llm.service already is llama-server.
- Do not implement llama.cpp --model-draft with a Hailo HEF. Draft and target must be two GGUFs in one process.
- Do not put Qwen3.8-2B-Distill on Hailo. The Hailo GenAI zoo does not compile Qwen3.5 / Gated DeltaNet.
- Do not use deepseek_r1_distill_qwen:1.5b on hailo-ollama (template corruption and crashes).
- Do not make Hailo’s sketch authoritative for balances, channel points, or “please pay this invoice.”
- Do not bind hailo-ollama or llama-server off localhost.
- Do not raise mercury-llm to --threads 4 or --ctx-size 32768 while this path is live. That configuration already produced 1.7 GiB zram use on mercury2.
- Do not let the AI layer execute pay, send, channel open/close, fee set, or magma buy. Prepare and recommend only.
- Remote DGX Ollama remains an optional last-resort fallback, not part of the local split. This spec can ship with DGX disabled.

3. Current reference deployment

Source of truth: docs/current-state.md, docs/architecture.md, docs/hardware-bom.md, scripts/systemd/mercury-llm.service, src/cli/mercury_cli/config.py. Snapshot date 2026-09-02.

Layer	Live contract
Host	Raspberry Pi 5-class ARM64, 16 GiB, Debian 13 (trixie)
SoC	4× Cortex-A76, kernel 6.12.47+rpt-rpi-2712

Layer	Live contract
NPU	Hailo-10H PCIe, firmware 5.1.1 — present, unused by LLM path
CPU model	Qwen3.8-2B-Distill-Q4_K_M.gguf via native llama-server
LLM bind	127.0.0.1:8089, OpenAI /v1/chat/completions
Unit flags	--ctx-size 32768 --threads 4 --reasoning off (must change)
Agent	FastAPI 0.5.5 on 127.0.0.1:8088
Money	Bitcoin Core 30.2.0 + LND 0.20.1-beta
Support	NBXplorer, BTCPay Server, nginx
Storage	Root ~1.9 TB; chain /mnt/blockchain ~1.8 TB Extreme 55DD
Observed risk	67.75 °C, historical throttle flag, 1.7/2 GiB zram used

CLI already implements `_ask_local()` against llama-server and `_ask_dgx()` against a remote Ollama URL (default 10.0.0.79:11434, qwen2.5:14b). The split replaces the primary path. DGX stays behind an explicit env opt-in.

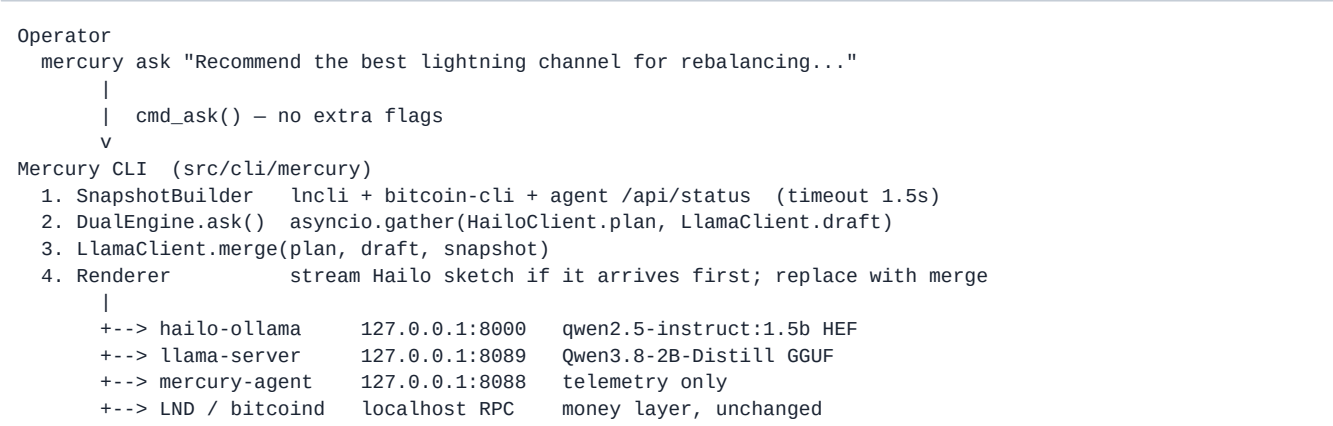
4. Why a split is required

Pi 5 LPDDR4X bandwidth is about 17 GB/s. Decode tokens/sec is bounded by resident weights plus KV cache. A 1.3 GB Q4 model cannot beat ~13 tok/s on the SoC even in isolation. Independent measurements on Pi 5 + Hailo-10H show the NPU is often slower than CPU on the same 1.5B-class model for raw decode, but much faster to first token and it leaves host RAM and CPU free.

Backend	Typical decode	TTFT	Host cost during generate
Hailo qwen2.5-instruct 1.5B HEF	~7–10 tok/s	~300 ms warm	~0.7 GB host RAM; A76s free
CPU Qwen3.8-2B-Distill Q4	~6–12 tok/s tuned	~1.5–3 s	SoC RAM + 2 pinned cores
CPU Gemma 3 1B (optional sprint)	~18–22 tok/s	low	faster, weaker Lightning language

Hailo therefore owns planning and TTFT. CPU owns numbers and the paragraph the operator reads. Together they hide the slowness of either path alone: Hailo paints structure immediately; CPU finishes the recommendation.

5. Target architecture



5.1 Process map

Unit	User	Bind	Model / job
hailo-ollama.service (new)	hailo-ollama	127.0.0.1:8000	HEF planner; KEEP_ALIVE=-1
mercury-llm.service	mercury-llm (not root)	127.0.0.1:8089	2B draft + merge; 2 threads, 8k ctx
mercury-agent.service	lnd	127.0.0.1:8088	status / channels / tools prepare
bitcoind / lnd	bitcoin / lnd	RPC localhost	authoritative ledger

5.2 Logical stages of one ask

Stage	Where	Budget	Output
S0 Snapshot	CLI subprocess / agent GET	≤ 1.5 s hard timeout	compact JSON ≤ 4 KB
S1a Plan	Hailo :8000	≤ 2.5 s, max 200 tokens	Plan JSON schema below
S1b Draft	CPU :8089	≤ 8 s, max 256 tokens	operator prose, no think block
S2 Merge	CPU :8089	≤ 5 s, max 256 tokens	single final answer
S3 Render	CLI stdout	starts at first token	one answer, one model label

S1a and S1b start together. S2 starts when both complete, or when one side hits timeout and the other has a usable body. S3 may begin streaming S1a.answer_sketch so the terminal is not silent.

6. Runners, ports, and models

6.1 Chosen stack

Question	Decision	Rationale
CPU runner	Existing llama-server	Already OpenAI-compatible; no second GGUF host
Hailo runner	hailo-ollama	Only supported GenAI HTTP surface for Hailo-10H
CPU model	Qwen3.8-2B-Distill-Q4_K_M.gguf	Current contract; Qwen-family merge
Hailo model	qwen2.5-instruct:1.5b	Official HEF; same family; most stable instruct target
Hailo speed alt	llama3.2:1b	Routing / classification only, not default planner
Forbidden Hailo	deepseek_r1_distill_qwen:1.5b	Known crash / template bugs
Optional sprint CPU	Gemma 3 1B on a second port	Out of scope for v1

6.2 Configuration surface

Extend src/cli/mercury_cli/config.py. Do not hard-code ports in cmd_ask.

```
LOCAL_LLM_URL      = "http://127.0.0.1:8089"          # existing
LOCAL_LLM_MODEL     = "/opt/mercury/models/Qwen3.8-2B-Distill-Q4_K_M.gguf"
HAILO_URL           = "http://127.0.0.1:8000"         # new
HAILO_MODEL         = "qwen2.5-instruct:1.5b"         # new
OLLAMA_URL          = "http://10.0.0.79:11434"        # last-resort only
MERCURY_SPLIT_AI    = "1" # default on once hailo-ollama is healthy
MERCURY_AI_DEBUG    = "0" # 1 prints plan/draft timings to stderr
MERCURY_ALLOW_DGX   = "0" # must be explicit to use remote fallback
```

6.3 HTTP dialects

Hailo-ollama speaks Ollama: POST /api/chat and POST /api/generate, GET /api/tags. Do not send a tools array unless a later milestone proves hailo-ollama accepts it; current builds error on tools. llama-server speaks OpenAI: POST /v1/chat/completions, GET /v1/models. Keep two thin clients. Do not pretend they are the same object.

7. Prompt contracts

Prompts are versioned strings in `src/cli/mercury_cli/prompts.py`. Changing tone later must not require touching HTTP code. All three prompts forbid chain-of-thought visible to the operator. `mercury-llm.service` already passes `--reasoning off`; the prompts must still say so, because this distill likes `<think>` blocks.

7.1 PLAN_SYS (Hailo)

```
PLAN_SYS = """You are Mercury's on-box planner for a Bitcoin Lightning node.
Return ONLY compact JSON. No markdown. No <think> block. No apology.
Schema:
{
  "intent": "rebalance|liquidity|status|pay|invoice|other",
  "need_tools": ["channels", "payments", "routing"],
  "risks": ["string"],
  "bullets": ["max 5 short factual bullets"],
  "answer_sketch": "one or two sentences the operator could read first",
  "recommended_action": "observe|recommend|requires_human_confirm"
}
Rules:
- Treat SNAPSHOT numbers as ground truth.
- Never invent a channel point, alias, sat amount, or peer pubkey.
- If SNAPSHOT is empty or stale, set risks accordingly and recommended_action=observe.
- recommended_action=requires_human_confirm for anything that would move funds.
- Max 200 tokens."""
```

7.2 DRAFT_SYS (CPU first pass)

```
DRAFT_SYS = """You are Mercury, local operator assistant on this Lightning node.
Write a concise recommendation an operator can act on after confirmation.
Rules:
- Use only SNAPSHOT facts for amounts, aliases, channel points, balances.
- If data is missing, say what command would fill it (mercury channels, mercury routing).
- Do not open, close, or pay anything. Do not output bolt11 or addresses to send to.
- No <think> block. No preamble like "Sure" or "As an AI".
- Target 120-220 words."""
```

7.3 MERGE_SYS (CPU second pass)

```
MERGE_SYS = """Reconcile PLAN JSON and DRAFT prose for the same operator question.
Priority: SNAPSHOT facts > DRAFT numbers > PLAN bullets.
Keep DRAFT voice. Adopt PLAN structure when it is clearer.
Drop any claim not supported by SNAPSHOT.
If PLAN and DRAFT disagree on a number, use SNAPSHOT or omit the number.
End with at most three next steps, each a Mercury CLI command where possible.
No <think> block. No mention of Hailo, llama, models, or dual engines."""
```

7.4 User payload shape

```
{
  "question": "<raw operator sentence>",
  "snapshot": { ... SnapshotBuilder output ... }
}
```

Send snapshot once per call as a fenced JSON block in the user message, not as a second system prompt. Hailo context windows on HEFs are small (often 2k–8k). SnapshotBuilder must already be compact.

7.5 Plan JSON schema (normative)

If Hailo returns non-JSON, DualEngine treats S1a as failed and merge becomes “draft only.” Do not regex-repair model prose into fake JSON beyond a single extract-first-{...}-last-} attempt.

8. Parallel control flow and UX

8.1 Happy path timeline

```
t=0.00 operator hits enter
t=0.05 snapshot start (agent /api/status + lncli listchannels parallel)
t=0.40 snapshot ready (cached agent payload often faster)
t=0.40 S1a Hailo POST /api/chat AND S1b CPU POST /v1/chat/completions
t=0.75 first Hailo tokens → optional quiet preamble not shown in v1
t=1.10 Hailo plan JSON complete
t=3.50 CPU draft complete (overlap hid Hailo wait)
t=3.50 S2 merge POST (CPU already warm)
t=5.50 final answer printed
```

Operator perception: ~5-6 s to a finished recommendation on a warm box, with no mention of two models. Cold HEF load is the only long stall; KEEP_ALIVE=-1 exists to prevent that after boot.

8.2 Streaming policy (v1)

v1 prints the final merge only. That is the simplest way to keep UX unnoticed. Do not interleave plan JSON onto the terminal. If merge exceeds 2 s and draft already exists, DualEngine MAY print draft immediately and then print a single-line revision only when merge differs materially (token-level diff > 40%). Default: wait for merge with a one-line “Thinking...” that is erased via carriage return. No backend names.

8.3 Concurrency primitive

Use asyncio + asyncio.to_thread around blocking urllib, or httpx.AsyncClient. The existing CLI uses urllib synchronously. Do not block the event loop on two serial urlopens. Recommended: add httpx to the CLI extra, keep urllib as fallback. Two threads is enough; do not spawn a process pool.

8.4 Cancellation and timeouts

Call	Timeout	On expiry
Snapshot	1.5 s	empty snapshot + risk flag; still run models
Hailo plan	2.5 s	S2 = draft only
CPU draft	8 s	if plan valid, S2 = expand plan into prose
CPU merge	5 s	print draft; never hang the CLI
Total ask	12 s	print best available + “partial” on stderr if DEBUG

9. Snapshot builder

Both models must see the same facts. Build snapshot once. Cap at 4 KB serialized. Prefer mercury-agent GET /api/status plus GET /api/channels. Fall back to lncli if agent is down. Never include macaroons, seeds, or RPC passwords.

```
SNAPSHOT_SCHEMA = {
  "ts": "2026-09-02T22:21:00-04:00",
  "chain": {"height": 965239, "synced": true},
```

```

"wallet": {"confirmed_sat": 0, "unconfirmed_sat": 0},
"channels": [
  {
    "alias": "peer-name",
    "chan_id": "...",
    "capacity_sat": 0,
    "local_sat": 0,
    "remote_sat": 0,
    "local_pct": 0,
    "active": true,
    "initiator": false
  }
],
"totals": {
  "active": 0, "local_sat": 0, "remote_sat": 0,
  "inbound_pct": 0, "outbound_pct": 0
},
"notes": ["zero_channels"] # machine flags, not prose
}

```

For the example prompt (“best channel for rebalancing inbound and outbound”), the interesting fields are `local_pct` per channel and `totals.inbound_pct`. If `active == 0`, planner intent stays “liquidity” but `recommended_action` becomes `observe`, and draft must say there is no channel to rebalance until mercury channel open.

10. Sample libraries and CLI integration

Proposed package: `src/cli/mercury_cli/ai/`. Keep `src/cli/mercury` as the `argparse` facade. New modules must be unit-testable without LND.

10.1 Module layout

```

src/cli/mercury_cli/
config.py           # add HAILO_* and flags
prompts.py         # PLAN_SYS / DRAFT_SYS / MERGE_SYS
ai/
  __init__.py
  snapshot.py       # SnapshotBuilder
  hailo_client.py   # Ollama dialect
  llama_client.py   # OpenAI dialect
  engine.py         # DualEngine.ask()
  render.py         # eraseable status + final print

```

10.2 hailo_client.py

```

from __future__ import annotations
import json
from typing import Any
import httpx

class HailoClient:
    def __init__(self, base: str, model: str, timeout: float = 2.5) -> None:
        self.base = base.rstrip("/")
        self.model = model
        self.timeout = timeout

    def healthy(self) -> bool:
        try:
            r = httpx.get(f"{self.base}/api/tags", timeout=0.4)
            return r.status_code == 200
        except httpx.HTTPError:
            return False

    async def chat(self, messages: list[dict[str, str]], num_predict: int = 200) -> str:
        payload = {

```



```

        "model": self.model,
        "messages": messages,
        "stream": False,
        "options": {"num_predict": num_predict, "temperature": 0.2},
    }
    async with httpx.AsyncClient(timeout=self.timeout) as client:
        r = await client.post(f"{self.base}/api/chat", json=payload)
        r.raise_for_status()
        data = r.json()
        return (data.get("message") or {}).get("content") or data.get("response") or ""

    @staticmethod
    def parse_plan(text: str) -> dict[str, Any] | None:
        start, end = text.find("{"), text.rfind("}")
        if start < 0 or end <= start:
            return None
        try:
            obj = json.loads(text[start:end + 1])
        except json.JSONDecodeError:
            return None
        if not isinstance(obj, dict) or "intent" not in obj:
            return None
        return obj

```

10.3 llama_client.py

```

from __future__ import annotations
import httpx

class LlamaClient:
    def __init__(self, base: str, model: str, timeout: float = 8.0) -> None:
        self.base = base.rstrip("/")
        self.model = model
        self.timeout = timeout

    def healthy(self) -> bool:
        try:
            r = httpx.get(f"{self.base}/v1/models", timeout=0.4)
            return r.status_code == 200
        except httpx.HTTPError:
            return False

    async def chat(self, messages: list[dict[str, str]], max_tokens: int = 256) -> str:
        payload = {
            "model": self.model,
            "messages": messages,
            "stream": False,
            "max_tokens": max_tokens,
            "temperature": 0.3,
        }
        async with httpx.AsyncClient(timeout=self.timeout) as client:
            r = await client.post(f"{self.base}/v1/chat/completions", json=payload)
            r.raise_for_status()
            data = r.json()
            return data["choices"][0]["message"]["content"]

```

10.4 engine.py — the gather

```

from __future__ import annotations
import asyncio
import json
from dataclasses import dataclass
from typing import Any

from mercury_cli.prompts import PLAN_SYS, DRAFT_SYS, MERGE_SYS

@dataclass
class AskResult:

```

```

text: str
source: str          # "merge" | "draft" | "plan" | "unavailable"
plan: dict[str, Any] | None
timings_ms: dict[str, int]

class DualEngine:
    def __init__(self, hailo, llama, debug: bool = False) -> None:
        self.hailo = hailo
        self.llama = llama
        self.debug = debug

    async def ask(self, question: str, snapshot: dict[str, Any]) -> AskResult:
        user = self._user(question, snapshot)
        plan_t = asyncio.create_task(self._plan(user))
        draft_t = asyncio.create_task(self._draft(user))
        plan, draft = await asyncio.gather(plan_t, draft_t, return_exceptions=True)

        plan_obj = plan if isinstance(plan, dict) else None
        draft_txt = draft if isinstance(draft, str) and draft.strip() else None

        if plan_obj and draft_txt:
            merged = await self._merge(question, snapshot, plan_obj, draft_txt)
            if merged:
                return AskResult(merged, "merge", plan_obj, {})
            return AskResult(draft_txt, "draft", plan_obj, {})
        if draft_txt:
            return AskResult(draft_txt, "draft", plan_obj, {})
        if plan_obj:
            sketch = plan_obj.get("answer_sketch") or json.dumps(plan_obj)
            return AskResult(str(sketch), "plan", plan_obj, {})
        return AskResult("Mercury AI unavailable (local engines down).", "unavailable", None, {})

    async def _plan(self, user: str) -> dict[str, Any] | None:
        raw = await self.hailo.chat(
            [{"role": "system", "content": PLAN_SYS},
             {"role": "user", "content": user}],
            num_predict=200,
        )
        return self.hailo.parse_plan(raw)

    async def _draft(self, user: str) -> str:
        return await self.llama.chat(
            [{"role": "system", "content": DRAFT_SYS},
             {"role": "user", "content": user}],
            max_tokens=256,
        )

    async def _merge(self, question, snapshot, plan, draft) -> str:
        user = (
            f"QUESTION:\n{question}\n\n"
            f"SNAPSHOT:\n{json.dumps(snapshot, separators=(',', ':'))}\n\n"
            f"PLAN:\n{json.dumps(plan, separators=(',', ':'))}\n\n"
            f"DRAFT:\n{draft}"
        )
        try:
            return await self.llama.chat(
                [{"role": "system", "content": MERGE_SYS},
                 {"role": "user", "content": user}],
                max_tokens=256,
            )
        except Exception:
            return ""

    @staticmethod
    def _user(question: str, snapshot: dict[str, Any]) -> str:
        snap = json.dumps(snapshot, separators=(",", ":"))
        return f"QUESTION:\n{question}\n\nSNAPSHOT:\n{snap}"

```

10.5 cmd_ask facade

```
def cmd_ask(ns) -> int:
    question = " ".join(ns.question).strip()
    cfg = load_config()
    snapshot = SnapshotBuilder(cfg).build()
    hailo = HailoClient(cfg.hailo_url, cfg.hailo_model)
    llama = LlamaClient(cfg.local_llm_url, "qwen3.8-2b")
    engine = DualEngine(hailo, llama, debug=cfg.ai_debug)

    if not hailo.healthy() and not llama.healthy():
        if cfg.allow_dgx:
            return _ask_dgx(ns)          # existing path, opt-in only
        print("Mercury AI unavailable.")
        return 1

    result = asyncio.run(engine.ask(question, snapshot))
    _print_ai_answer(result.text)        # existing renderer; one label: "Local"
    return 0 if result.source != "unavailable" else 1
```

Health checks are 400 ms and run once per invocation, not per stage. If Hailo is down, DualEngine still runs draft+merge-skip. If llama is down and Hailo is up, print answer_sketch plus bullets — degraded but usable. Never block on DGX unless MERCURY_ALLOW_DGX=1.

10.6 Dependencies

Add httpx to src/cli extras or the agent requirements if the CLI shares the venv. Pin a current version already used elsewhere if present. Tests may use respx or a local httpx mock transport. Do not add langchain, llama-index, or an agent framework.

11. systemd, affinity, and thermal budget

11.1 mercury-llm.service (change)

```
[Service]
User=mercury-llm
Group=mercury-llm
CPUAffinity=2 3
Environment=LD_LIBRARY_PATH=/opt/mercury/llama.cpp/lib
ExecStart=/opt/mercury/llama.cpp/bin/llama-server \
    -m /opt/mercury/models/Qwen3.8-2B-Distill-Q4_K_M.gguf \
    --host 127.0.0.1 --port 8089 \
    --ctx-size 8192 --threads 2 --reasoning off
# drop User=root; drop ctx 32768; drop threads 4
```

11.2 hailo-ollama.service (new)

```
[Unit]
Description=Mercury Hailo planner (hailo-ollama)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=hailo-ollama
Group=hailo-ollama
Environment=OLLAMA_HOST=127.0.0.1:8000
Environment=OLLAMA_KEEP_ALIVE=-1
ExecStart=/usr/bin/hailo-ollama serve
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Confirm the hailo-ollama binary path against the 5.1.1 package on the reference host. Bind 127.0.0.1 only. Pre-pull qwen2.5-instruct:1.5b during install so first ask is warm. PCIe Gen 3 must be enabled (raspi-config / dtparam=pcie1_gen=3) or Hailo TTFT regresses.

11.3 Why two CPU threads

The live host already runs bitcoind, lnd, nbxplorer, btcpayserver, nginx, mercury-agent, and hve-lifeos. Four llama threads plus those daemons is how zram filled. Pin llama to cores 2–3. Leave 0–1 for the money layer. Hailo does not need a CPUAffinity line.

11.4 Thermals

mercury2 was 67.75 °C with a historical throttle flag. Dual infer will add heat. Acceptance test must log vcgencmd get_throttled before and after a 10-ask burst. If throttle bits flip, stop the burst and treat cooling as a blocker, not a model problem.

12. Failure modes and fallbacks

Fault	Operator sees	Engine does
Hailo down / HEF missing	Normal local answer, slightly slower TTFT	draft only; source=draft
llama-server down	Short structured note from plan	plan sketch; source=plan
Both down, DGX off	Mercury AI unavailable.	exit 1
Both down, DGX on	Existing remote answer	_ask_dgx only if ALLOW_DGX=1
Hailo non-JSON	Normal draft	parse_plan → None
Merge timeout	Draft text	source=draft
Snapshot timeout	Answer that admits missing channel data	empty snapshot + notes
Zero channels (current live state)	You have no active channels to rebalance	intent=liquidity, action=observe
Model invents a peer	Must not survive merge	MERGE_SYS + snapshot priority

13. Security and money-layer isolation

Unchanged from docs/architecture.md and SECURITY.md. Dual-engine reasoning is still advisory.

- No new public port. 8000 and 8089 stay on 127.0.0.1.
- hailo-ollama does not run as root. mercury-llm must stop running as root as part of this work.
- Snapshot contains no seed, wallet password, macaroon path contents, or backup token.
- PLAN.recommended_action=requires_human_confirm is informational. The CLI must still use existing confirmation prompts for pay/send/channel/magma buy.
- Do not auto-invoke POST /api/tools/{name}/execute from DualEngine.
- Logs must not dump full snapshots at info level (channel graph + balances). Debug only.
- Compromise of hailo-ollama must not imply compromise of LND macaroons. Separate users.

14. Files to touch

Path	Change
src/cli/mercury_cli/config.py	HAILO_URL/MODEL, flags, from_env
src/cli/mercury_cli/prompts.py	new file, three system prompts
src/cli/mercury_cli/ai/*.py	new package as in §10
src/cli/mercury	cmd_ask and _ai_insight call DualEngine
scripts/systemd/mercury-llm.service	threads 2, ctx 8192, drop root
scripts/systemd/hailo-ollama.service	new
docs/architecture.md	dual-engine diagram; retire Hailo-unused wording
docs/current-state.md	after deploy, record hailo-ollama health
docs/hardware-bom.md	Hailo now in the inference path
docs/roadmap.md	check dual-engine item
tests/test_dual_engine.py	new: parse_plan, gather fallbacks, merge skip
tests/test_config_consistency.py	new env keys
src/agent/requirements or CLI deps	httpx if not present

install.sh still must not silently install Hailo drivers or models (current installer limitation). Document a manual appendix command block for hailo-h10-all + model pull.

15. Implementation plan for Luna

Phase 0 — Recon (half day)

- Read this spec and docs/{architecture,current-state,hardware-bom,roadmap}.md.
- On a non-prod clone or the reference host in a maintenance window: confirm hailortcli fw-control identify, ss -lntp for 8088/8089, and whether hailo-ollama is installed.
- Do not change mercury2 money-layer units in Phase 0.

Phase 1 — CPU unit hygiene (half day)

- Ship mercury-llm.service changes on a branch. Verify Qwen3.8-2B still loads at ctx 8192.
- Create mercury-llm user; prove llama-server does not need root.
- Record tok/s with llama-bench or a fixed 50-token prompt before/after thread change.

Phase 2 — Hailo service (one day)

- Install hailo-ollama if missing; pull qwen2.5-instruct:1.5b; systemd unit; localhost bind.
- curl /api/tags and a 20-token /api/chat smoke test.
- Reject DeepSeek HEF in any default list.

Phase 3 — Library + tests (one to two days)

- Land mercury_cli/ai and prompts with mocked HTTP. Tests in §16 must pass offline.
- Wire cmd_ask. Default label remains “Local”.
- MERCURY_AI_DEBUG=1 prints timings to stderr only.

Phase 4 — On-box soak (half day)

- Ten identical asks including the rebalance sentence. No zram growth, no throttle bit change.
- Repeat with hailo-ollama stopped (draft only) and with llama-server stopped (plan only).
- Confirm pay/channel commands still require the existing confirmation prompt and do not call DualEngine.execute.

Phase 5 — Docs PR

Update architecture and current-state in the same PR as the code. Do not merge code that leaves docs claiming Hailo is unused.

16. Acceptance tests

16.1 Offline (CI)

- parse_plan accepts exact schema JSON and rejects markdown-wrapped garbage after one extract attempt fails.
- ask() with Hailo exception + valid draft returns source=draft.
- ask() with llama exception + valid plan returns source=plan and uses answer_sketch.
- ask() with both exceptions returns source=unavailable.
- Merge HTTP 500 returns draft, not unavailable.
- Snapshot serializer stays ≤ 4 KB on a fixture of 20 channels.
- config.from_env reads MERCURY_HAILO_URL and MERCURY_ALLOW_DGX.

16.2 On-box

- Command: mercury ask "Recommend the best lightning channel for rebalancing inbound and outbound liquidity"
- Exit 0 when at least one engine is up.
- Stdout contains no words: Hailo, llama, GGUF, HEF, planner, merge, DGX, Ollama — unless DEBUG.
- If snapshot.totals.active == 0, answer states there is no channel to rebalance.
- If two unbalanced channels exist, answer names aliases/chan_ids from snapshot only.
- Wall time warm-warm p50 ≤ 8 s, p95 ≤ 12 s on the reference Pi 5.
- After 10 asks: swap used does not climb more than 100 MiB; get_throttled bits unchanged.
- ss -lntp shows 8000 and 8089 on 127.0.0.1 only.

17. Worked example

Operator types:

```
mercury ask "Recommend the best lightning channel for rebalancing inbound and outbound liquidity"
```

Snapshot (abridged, illustrative — live mercury2 currently has zero channels):

```
{
  "chain": {"height": 965239, "synced": true},
  "totals": {"active": 2, "local_sat": 850000, "remote_sat": 150000, "outbound_pct": 85},
  "channels": [
    {
      "alias": "ACINQ", "local_pct": 92, "local_sat": 460000, "remote_sat": 40000, "active": true,
      "alias": "Olympus", "local_pct": 78, "local_sat": 390000, "remote_sat": 110000, "active": true
    }
  ]
}
```

Hailo plan (never printed raw):

```
{
  "intent": "rebalance",
  "need_tools": ["channels", "routing"],
  "risks": ["rebalance spends routing fees", "do not force close"],
  "bullets": ["ACINQ is 92% local", "Olympus is 78% local", "node outbound-heavy"],
  "answer_sketch": "Push sats out of ACINQ first; it is the most outbound-skewed.",
  "recommended_action": "recommend"
}
```

CPU draft (never printed if merge succeeds):

ACINQ is 92 percent local (460k of 500k). Olympus is healthier but still outbound. A circular rebalance that spends from ACINQ toward a peer with inbound capacity is the first experiment. Use mercury rebalance --dry-run before any real payment. Do not close ACINQ to fix this.

What the operator actually reads:

This node is outbound-heavy (about 85 percent local). ACINQ is the worst skew at 92 percent local and is the first channel to rebalance away from. Olympus is the secondary candidate. Dry-run first: mercury rebalance --dry-run. Do not close a channel to fix liquidity. Confirm any real rebalance payment yourself.

If the live snapshot stays at zero channels, both engines must produce the observe path: there is no channel to rebalance; opening inbound is a human channel-open decision, not an AI payment.

18. Open questions left to Luna

- Exact hailo-ollama binary and package name on Debian 13 with hailo-h10-all 5.1.1 (serve vs bare hailo-ollama).
- Whether llama-server model name in /v1/chat/completions must be the GGUF basename or an alias; probe GET /v1/models on the live unit.
- Whether httpx is acceptable in the CLI runtime or urllib+ThreadPoolExecutor is required to avoid a new dependency.
- Whether _ai_insight() for status --ai should use DualEngine or Hailo-only (recommendation: Hailo-only for --ai one-liners, DualEngine for ask).
- User/group creation convention (mercury-llm, hailo-ollama) vs reusing existing service users.
- If PCIe Gen 3 is already set on mercury2; if not, schedule with the bootloader update already flagged in current-state.md.

End of MN-ARCH-REASON-001. Implement against mercury-node main. Do not revive the retired humanvalueexchange/mercury Hailo-8L / Phi-3.5 path. Do not spend time on MCP, agent cards, or public registry in this milestone.